



MINISTERIO DE
EDUCACIÓN
CIENCIA Y
TECNOLOGÍA

**ESCUELA ESPECIALIZADA EN INGENIERÍA ITCA-FEPADE, MEGATEC, LA
UNIÓN**

ESCUELA DE INGENIERÍA DE COMPUTACIÓN

TÉCNICO EN INGENIERÍA DE DESARROLLO DE SOFTWARE

Módulo:

Desarrollo de Aplicaciones Móviles

Tema:

Interrogantes Semana 2

Sección:

DES2-01

Integrantes:

Herrera Guevara, Erick Anibal - 048325

1. ¿Qué diferencia existe entre var, dynamic, final y const?

Var: Inferir el tipo de dato automáticamente a partir del valor inicial. Una vez inferido, el tipo queda fijado estáticamente y no puede cambiar a otro tipo durante la ejecución.

Dynamic: Desactiva la comprobación estática del compilador. La variable puede cambiar tanto de valor como de tipo de dato en cualquier momento, transfiriendo el riesgo de error al tiempo de ejecución.

Final: Define una variable inmutable que se inicializa una sola vez durante la ejecución. No puede reasignarse posteriormente.

Const: Define una constante inmutable cuyo valor debe ser conocido de forma estricta en tiempo de compilación.

2. ¿Qué error evita null safety y qué alternativas existen antes de utilizar !?

Evita las excepciones en tiempo de ejecución causadas por intentar acceder a propiedades o invocar métodos sobre referencias nulas "null", haciendo que los tipos sean no anulables por defecto.

Se pueden usar alternativas como:

- Acceso condicional "?": Ejecuta la operación únicamente si la variable no es nula.
- Operación de coalescencia nula "??": Proporciona un valor alternativo o por defecto si la expresión resulta ser null.
- Asignación alternativa "??=": Asigna un valor solo cuando la variable es nula.
- Parámetro **required**: Obliga a entregar el dato en constructores o funciones.
- Validación condicional previa con if.

3. ¿Cuándo una condición debería convertirse en una función?

Debe convertirse en una función cuando la lógica de validación o evaluación se repite en distintas partes del código.

Cuando la regla de negocio es compleja o requiere múltiples pasos, y al encapsularla mejora la legibilidad e intención del programa.

También cuando se desea reducir la oxidación excesiva de bloques if/else en el flujo principal.

4. ¿Qué ventaja aporta List<Producto> frente a List<Map<String, dynamic>> dentro del programa?

Tiene ventajas como seguridad en compilación porque el compilador detecta errores antes de la ejecución si se intenta acceder a una propiedad inexistente o asignar un tipo incompatible.

Prevención de errores de escritura, elimina los errores silenciosos provocados por tipos al escribir claves en cadenas de texto.

Permite encapsulación y reglas de negocio agrupando métodos, constructores con validaciones internas y getters calculados en la clase.

5. ¿Cuándo conviene utilizar herencia y cuándo es preferible la composición?

La herencia conviene utilizarla únicamente cuando existe una relación estable del tipo (es un) is-a y la subclase realmente reutiliza y especializa el comportamiento de una clase base común.

La composición es preferible cuando la relación es del tipo (tiene un) has-a o cuando una clase colabora con otros objetos combinándolos. Ofrece diseños muchos mas flexible, evita el acoplamiento rigido de jerarquías profundas y es el patrón arquitectónico principal en Flutter.

6. ¿Cómo permite el polimorfismo eliminar condicionales basados en el tipo concreto del objeto?

El polimorfismo permite agrupar diferentes subclases bajo una misma clase base o interfaz común por ejemplo, List<Empleado>. Cuando se invoca un método como empleado.calcularPago(), cada objeto ejecuta automáticamente su propia implementación sobrescrita (@override)

7. ¿Qué lógica conviene mantener fuera de main() y por qué?

La funcion main() debe actuar únicamente como el punto de entrada del programa y limitarse a coordinar el flujo de datos, invocar funciones y presentar los resultados finales.

Los que debe extraer fuerza del main(), son calculos, operaciones aritmeticas, procesamiento de colecciones y funciones puras.

Reglas de negocio con validaciones de datos y condiciones complejas.

¿Por qué? Mantenemos la separación de responsabilidad, evitando funciones sobrecargadas que mezclan entrada de datos, cálculo y presentación.

También hay una reutilización directa en Flutter, la misma lógica funciones, clases de dominio pueden integrarse en widgets, controladores o servicios móviles sin tener que reescribir el código.